BZAN 554: Training Deep Neural Networks

Slide-by-slide study notes for Michel Ballings's **deep.pdf** (105 slides). Slide numbers match the PDF pages. Outline slides are retained so you can follow the deck exactly. **Clarification** labels identify notation errors or qualifications; other notes explain the lecture's content. Code is cleaned up for readability and illustrates the slides; it has not been executed.

Topic map

Slides	Topic
1-4	Introduction and embeddings
5-7	Backpropagation
8-17‘	Optimizer categories, momentum, Nesterov
18-22	Learning rate scheduling
23-37	AdaGrad, RMSprop, Adam
38-59	Gradient problems and weight initialization
60-61	Nonsaturating activations
62-75	Batch normalization
76-78	Gradient clipping
79-91	Regularization, early stopping, L1/L2
92-99	Dropout
100-101	Data augmentation
102-105	Depth versus width

Notation

- `J` or `L`: loss; `theta`: model parameters; `eta`: learning rate; `t`: update index.
- `d_t = gradient J(theta_t)`: gradient at step `t`. Vector squares, roots, and divisions in optimizer formulas act element by element.
- `fan_in`: inputs to a unit; `fan_out`: outputs connected to a unit.
- `z`: weighted sum before activation; `h`: activated output.
- Momentum symbols vary across the deck. Adam notes use the standard `m` for the first moment and `v` for the second moment to avoid the deck's inconsistent labels.

Slide 1 - Title

- Course: BZAN 554, Deep Learning for Business Applications; instructor: Michel Ballings.
- This deck explains how to train deep networks efficiently, keep gradients usable, limit overfitting, and choose depth versus width.

Slide 2 - Outline: embeddings

- The highlighted opening topic is embeddings.
- The overall sequence is representation, gradient computation, optimization, gradient stability, regularization, and network architecture.

Slide 3 - Embeddings: one categorical value

- An embedding maps a category ID to a learned dense vector using a lookup table.
- With 20 categories and 2 embedding dimensions, the table has shape `(20, 2)` and contains 40 learned values.
- A one-hot row multiplied by this table simply selects the category's row. The lookup avoids constructing and multiplying a mostly zero vector.

```
inputs = tf.keras.layers.Input(shape=(1,), dtype="int32")
embedding = tf.keras.layers.Embedding(input_dim=20, output_dim=2)(inputs)
```

- **Clarification:** an ordinary `Dense(2)` includes a bias by default, while the lookup has no separate bias. Exact equivalence requires accounting for that bias. A length-one input also retains a sequence dimension: embedding output is `(batch, 1, 2)`, versus `(batch, 2)` for the dense example.

Slide 4 - Embeddings: several categorical values

- A K-hot input can have several active categories. For `k=4`, pass four category IDs instead of a 20-column binary vector.
- `Embedding(20, 2)` returns one two-dimensional vector per ID, giving shape `(batch, 4, 2)`.
- **Clarification:** the lookup alone does not reproduce the K-hot dense result. Summing the four embedding vectors reproduces the linear weighted sum; concatenating or averaging represents a different operation.
- Encode categories with consecutive integers. `input_dim = maximum_ID + 1` includes zero-based indexing.
- Variable-length inputs can be padded; padding should be handled so it does not become a meaningful category accidentally.
- The slide's restriction concerns a direct lookup replacement for binary inputs: arbitrary numeric counts are not category IDs. A more elaborate weighted embedding sum could represent counts, but is beyond the example.
- Referenced class example: `embedding.py`.

Slide 5 - Outline: backpropagation

- Transition to efficient computation of gradients through the network.

Slide 6 - Backpropagation: training workflow diagram

1. Randomly initialize the model weights.
2. Feed inputs forward through the network to obtain predictions.
3. Compare predictions with desired outputs using a loss function.
4. Differentiate the loss with respect to the final output.
5. Backpropagate through the saved forward computations to obtain parameter gradients.
6. Use an optimizer to update the weights.
7. Repeat until the stopping criterion is met.

- **Key distinction:** backpropagation computes gradients; the optimizer decides how to use them to change weights.
- The diagram's stack/unstack idea means intermediate forward results are retained and reused during differentiation.

Slide 7 - Why backpropagation matters

- The chain rule connects the loss to parameters in earlier layers.
- Backpropagation reuses derivatives from later layers rather than independently recomputing each complete derivative path.
- This reuse makes gradient computation feasible for deep networks.
- The slide refers to a worked example in `backprop.pdf`; that separate file is not included in these notes.

Slide 8 - Outline: faster optimizers

- Transition from gradient calculation to methods that use gradients to improve convergence.

Slide 9 - Three optimizer categories

Category in the lecture	Meaning	Examples
Global, constant learning rate	Same base rate for all parameters and steps	Momentum, Nesterov
Global, adaptive learning rate	One shared rate changes across steps	Learning rate scheduling

Category in the lecture	Meaning	Examples
Local, adaptive learning rate	Effective rate changes separately for each parameter	AdaGrad, RMSprop, Adam

- Momentum changes the update direction and size through gradient history even when its base learning rate is constant.
- These categories describe the mechanisms; scheduling can also be combined with momentum or adaptive optimizers.

Slide 10 - Outline: global, constant learning rate

- Introduces momentum and Nesterov under the shared, constant base-rate category.

Slide 11 - Outline: momentum

- Focuses on momentum: add memory of previous gradients to the update.

Slide 12 - Momentum: plateaus and elongated valleys

- Different feature scales can produce a loss surface that is steep in one direction and flat in another.
- Plain SGD may zigzag across the steep direction while progressing slowly along the valley.
- The contour diagrams show momentum smoothing that zigzag and making faster progress toward the bottom.
- The rolling-ball analogy explains how previous motion can carry an optimizer through a region with a small gradient.

Slide 13 - Momentum equations

The lecture uses a weighted moving average:

```
nu_t = gamma * nu_(t-1) + (1 - gamma) * d_t
theta_(t+1) = theta_t - eta * nu_t
```

- `gamma` determines the amount of memory; the slide suggests `0.9`.
- Consistent gradients reinforce one another, while gradients that reverse direction tend to cancel.
- On a zero-gradient plateau, accumulated motion decays gradually instead of vanishing immediately.

- **Clarification:** Keras SGD uses an accumulated velocity convention without the lecture's `(1-gamma)` multiplier on the new gradient. The same numeric learning rate therefore does not make the two conventions identical. See Keras SGD documentation.

Slide 14 - Momentum in TensorFlow

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)
```

- `momentum=0` gives ordinary SGD; a positive value introduces gradient history.
- The values shown are the lecture's example, not a guarantee of suitable settings for every dataset.

Slide 15 - Outline: Nesterov's accelerated gradient

- Transition to a momentum method that anticipates where the accumulated motion is taking the parameters.

Slide 16 - Nesterov: look ahead before correcting

- Momentum already implies an approximate move of `-eta * gamma * nu_(t-1)`.
- Nesterov evaluates the gradient at that anticipated location rather than at the original location.

```
lookahead = theta_t - eta * gamma * nu_(t-1)
nu_t = gamma * nu_(t-1) + (1 - gamma) * gradient J(lookahead)
theta_(t+1) = theta_t - eta * nu_t
```

- The diagram contrasts a correction at the current position with a correction after the momentum move.
- Intuition: anticipate overshooting and adjust sooner.

Slide 17 - Nesterov in TensorFlow

```
optimizer = tf.keras.optimizers.SGD(
    learning_rate=0.1, momentum=0.9, nesterov=True
)
```

- The lecture argues that looking ahead often speeds training relative to ordinary momentum.
- Treat the performance claim as an empirical tendency, not a universal guarantee.
- The built-in implementation uses Keras's momentum convention; see the distinction on slide 13.

Slide 18 - Outline: global, adaptive learning rate

- Introduces changing the shared learning rate during training.

Slide 19 - Outline: learning rate scheduling

- Focuses on a schedule that reduces the base learning rate over time.

Slide 20 - Exponential learning rate scheduling

```
eta_t = eta_0 * r^(t / d)
```

- `eta_0`: initial rate; `r`: decay factor; `d`: decay interval; `t`: elapsed epochs in this example.
- If `0 < r < 1`, the learning rate decreases. Larger early steps allow progress; smaller later steps support finer adjustment.
- For `eta_0=0.1`, `r=0.95`, and `d=20`: the rate is `0.095` after 20 epochs and `0.09025` after 40 epochs.
- This is a continuous exponential schedule; it need not wait until an interval boundary to change.

Slide 21 - Learning rate decay plot

- The curve starts around `0.1` and declines smoothly across approximately 1,000 epochs.
- The same decay factor repeats multiplicatively; this is not subtraction of a fixed amount each epoch.
- For the preceding example, the rate at epoch 1,000 is approximately `0.00769`.

Slide 22 - Exponential decay in TensorFlow

```
schedule = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=0.1,
    decay_steps=10000,
    decay_rate=0.95,
)
optimizer = tf.keras.optimizers.SGD(learning_rate=schedule)
```

- **Study distinction:** the slide's conceptual example counts epochs; the optimizer schedule counts optimizer steps. A step is generally one mini-batch update, so 10,000 steps is not automatically 10,000 epochs.
- The schedule is supplied as the optimizer's `learning_rate`.

Slide 23 - Outline: local, adaptive learning rate

- Introduces learning rates that adapt separately for each parameter.

Slide 24 - Why adapt per parameter?

- A single rate cannot account for all differences in gradient scale or frequency.
- Each parameter may need a different amount of acceleration or damping as training progresses.
- AdaGrad, RMSprop, and Adam use gradient history to make this adjustment automatically.

Slide 25 - Outline: AdaGrad

- Focuses on adaptation using accumulated squared gradients.

Slide 26 - AdaGrad: frequent versus rare features

- A global decay schedule can make learning too slow for rare features before they have received enough informative updates.
- Keeping a simple count of nonzero observations would distinguish frequent and rare features, but does not capture the magnitude of small nonzero gradients.
- AdaGrad instead accumulates squared gradients: larger or more frequent gradients increase the denominator more.
- Parameters with little gradient history retain relatively larger effective learning rates.
- **Clarification:** the slide's convergence statement assumes a convex problem and appropriate learning-rate conditions. Deep-network losses are generally nonconvex; simply reducing a rate does not guarantee a global optimum.

Slide 27 - AdaGrad: contour diagram

- The diagram revisits the elongated bowl: ordinary GD tends to follow the steep direction and progress slowly along the flat direction.
- AdaGrad damps the steep direction more because it accumulates larger squared gradients there.
- This changes the relative sizes of the coordinate updates, producing a more direct path in the illustrated example.

Slide 28 - AdaGrad equations

Using `s` for the accumulator:

```
s_t = s_(t-1) + d_t^2
theta_(t+1) = theta_t - eta * d_t / sqrt(s_t + epsilon)
```

- There is one accumulator per parameter.
- Squaring prevents positive and negative gradients from canceling in the accumulator.
- The square root brings the denominator back to the scale of the gradient.
- `epsilon` prevents division by zero. The slide shows a diagonal matrix representation, but elementwise division expresses the same intended per-parameter scaling more simply.

Slide 29 - AdaGrad: strengths and weakness

- Automatically adjusts the effective learning rate at every update and separately for every parameter.
- Can speed training and reduce the amount of manual learning-rate tuning.
- Its accumulator never decreases, so the effective learning rate can become excessively small.
- This is its central limitation: distant gradient history is never forgotten.

Slide 30 - AdaGrad in TensorFlow

```
optimizer = tf.keras.optimizers.Adagrad(
    learning_rate=0.001,
    initial_accumulator_value=0.1,
    epsilon=1e-7,
)
```

- The initial accumulator gives a nonzero starting denominator.

- The lecture cautions that AdaGrad may stall too early when training neural networks.
- Original paper named in the slide: Duchi, Hazan, and Singer, *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization.*

Slide 31 - Outline: RMSprop

- Introduces a method that forgets old squared gradients instead of accumulating them forever.

Slide 32 - RMSprop equations

```
s_t = rho * s_(t-1) + (1 - rho) * d_t^2
theta_(t+1) = theta_t - eta * d_t / sqrt(s_t + epsilon)
```

- Replaces AdaGrad's cumulative sum with an exponentially weighted average of squared gradients.
- Old observations lose influence, preventing an ever-growing accumulator from automatically driving the rate toward zero.
- `rho` is called the decay rate. A larger value gives longer memory.
- **Clarification:** the slide's informal example omits the new-observation weight. Starting at zero, the first squared gradient contributes `(1-rho)*d_1^2` initially and `rho^(t-1)*(1-rho)*d_1^2` at step `t`.
- The slide mentions `0.999` in prose, while the next code example uses `0.9`; these are different settings, not the same value.

Slide 33 - RMSprop in TensorFlow

```
optimizer = tf.keras.optimizers.RMSprop(
    learning_rate=0.001, rho=0.9, momentum=0.0, epsilon=1e-7
)
```

- `rho` corresponds to the previous slide's decay coefficient.
- In this example, `momentum=0` means there is no additional momentum term.
- The lecture presents RMSprop as an improvement over AdaGrad for many neural-network problems and introduces Adam as the next combination.

Slide 34 - Outline: Adam

- Focuses on combining momentum-like direction estimation with adaptive gradient scaling.

Slide 35 - Adam: first and second moments

Use standard notation:

```
m_t = beta_1 * m_(t-1) + (1 - beta_1) * d_t
v_t = beta_2 * v_(t-1) + (1 - beta_2) * d_t^2
```

- `m_t` tracks the mean-like first raw moment and provides momentum.
- `v_t` tracks the second raw moment and determines coordinate scaling.
- **Clarification:** a second raw moment is a mean of squares, not the variance itself. Variance additionally subtracts the squared mean.
- The slide's gamma labels and listed values conflict. The conventional assignment is `beta_1=0.9` for the first moment and `beta_2=0.999` for the second. See Keras Adam documentation.
- The lecture describes Adam as state of the art; read that as the deck's characterization, not a claim that it universally outperforms all optimizers.

Slide 36 - Adam: bias correction

- Initialize `m_0=v_0=0`. Early moving averages underestimate the moments because their history starts at zero.

```
m_hat_t = m_t / (1 - beta_1^t)
v_hat_t = v_t / (1 - beta_2^t)
theta_(t+1) = theta_t - eta * m_hat_t / (sqrt(v_hat_t) + epsilon)
```

- The correction is strongest at the beginning. As `t` increases, the denominators approach 1.
- **Clarification:** corrections are still calculated later; their effect becomes negligible rather than abruptly disappearing.
- Keep uncorrected moving averages as persistent state; corrected versions are used for the update.
- This is the standard conceptual Adam formula. Implementations can use an equivalent rearrangement with a different epsilon convention.

Slide 37 - Adam pseudocode and TensorFlow

```
# Conceptual pseudocode; d is the current gradient vector.
m = 0.0
v = 0.0
for t in range(1, num_iterations + 1):
    d = compute_gradient()
    m = beta_1 * m + (1 - beta_1) * d
    v = beta_2 * v + (1 - beta_2) * d**2
    m_hat = m / (1 - beta_1**t)
    v_hat = v / (1 - beta_2**t)
    w = w - learning_rate * m_hat / (np.sqrt(v_hat) + epsilon)

optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-7
)
```

- **Correction to the slide's pseudocode:** start `t` at 1 to avoid a zero bias-correction denominator. Do not overwrite the persistent moment accumulators with bias-corrected values.
- Original paper: Kingma and Ba, *Adam: A Method for Stochastic Optimization*.

Slide 38 - Outline: vanishing/exploding gradients

- Transition to gradient stability, with four remedies: initialization, activations, batch normalization, and clipping.

Slide 39 - Outline: problem statement

- Begins the explanation of why deep networks can lose their learning signal.

Slide 40 - Variance preliminaries

- Assume standardized inputs and independent, zero-mean weights, each with variance 1.
- Under those assumptions, `Var(X_i*w_i)=Var(X_i)*Var(w_i)=1`.
- If the terms are uncorrelated, `Var(sum_i X_i*w_i)=sum_i Var(X_i*w_i)`.
- For `m` inputs, the preactivation therefore has variance `m`. More inputs can make its magnitude much larger.
- **Clarification:** independence is a sufficient condition for the product-variance calculation; uncorrelatedness alone does not generally imply the needed independence of squared terms. Covariance terms must be included when sum terms are correlated.

Slide 41 - Many inputs can cause saturation

- With 200 inputs under the preceding assumptions, `Var(z)=200` and `SD(z)=sqrt(200)=14.14`.
- Sigmoid and tanh outputs at such large magnitudes are near their limits, where derivatives are tiny.
- A gradient for an early weight contains a chain of factors, including the hidden activation derivative:

```
dE/dw_1 = (dE/do) * (do/dh_1) * (dh_1/dz_h1) * (dz_h1/dw_1)
```

- A very small factor can make the resulting parameter update very small.

Slide 42 - Saturating activation derivatives

```
sigmoid'(z) = sigmoid(z) * (1 - sigmoid(z))
tanh'(z) = 1 - tanh(z)^2
```

- At `z=14.14`, the slide gives approximately `7.23e-7` for sigmoid's derivative and `2.09e-12` for tanh's derivative.
- Large positive and negative inputs both produce near-zero derivatives.
- The issue is the activation's slope, not merely whether the output is close to a boundary.

Slide 43 - A modest increase in z can sharply reduce learning

- Sigmoid's derivative is about `0.105` at `z=2` and `0.0000454` at `z=10`.
- The latter is roughly 2,300 times smaller; the slide rounds the comparison to about 2,200.
- **Correction:** the slide calls these quantities `h`, but they are derivatives. The sigmoid output itself is approximately `0.881` at 2 and `0.999955` at 10.
- Holding the other derivative factors fixed, a smaller slope produces a proportionally smaller update.

Slide 44 - Why earlier layers receive weak gradients

- Backpropagation multiplies derivatives along the path from output to earlier layers.
- Repeated multiplication by small values can make early-layer gradients extremely small.
- In the lecture's funnel-shaped example, earlier layers also have more inputs and can saturate sooner under the assumed initialization.
- **Key takeaway:** earlier layers often receive the weakest learning signal. The fundamental issue is the chain of derivatives, not funnel shape alone.

Slide 45 - Outline: solution 1, weight initialization

- Transition to selecting weight distributions that preserve useful signal scales.

Slide 46 - Initialization scaled by the number of inputs

The lecture's preliminary normalized initialization is:

```
w ~ Uniform(-1/sqrt(m), 1/sqrt(m))
bias = 0
```

- Uniform variance is `(b-a)^2/12`, so each weight has variance `1/(3m)`.
- With unit-variance independent inputs, the weighted sum has variance `m * 1/(3m) = 1/3`.
- Its standard deviation is approximately `0.577`, versus `14.14` for 200 inputs with unit-variance weights.
- Scaling by `1/sqrt(m)` cancels the growth with input count and reduces initial saturation. It does not force every `z` into a fixed interval.

Slide 47 - Variance and the activation curve

- The sigmoid graph shows the central region with a useful slope and the flat tails with almost no slope.
- Low preactivation variance concentrates more observations near the center.
- High variance puts more observations into saturated tails, increasing the chance of weak gradients.

Slide 48 - Preliminary initialization does not solve everything

- The plot tracks tanh activation spread during learning under `Uniform(-1/sqrt(m), 1/sqrt(m))`.
- Marker-only curves show 98-percentile bounds; marked lines show standard deviations.
- The lecture interprets the early layers' growing spread toward activation limits as earlier saturation.
- Good initial scaling helps, but distributions can still drift during training.

Slide 49 - Activation histograms at the end of learning

- The figure averages activation histograms across units within a layer and 300 test examples.
- Large concentrations near `-1` and `+1` indicate tanh saturation; a distribution concentrated nearer zero is less saturated.
- The lecture emphasizes saturation in earlier layers.
- These graphs motivate initialization that considers both forward and backward propagation.

Slide 50 - Preserve forward and backward variance

- Glorot and Bengio's goal is to keep activations from shrinking or exploding as they move forward.
- They also aim to keep gradient variance stable as gradients move backward.
- In the idealized derivation, neighboring layers should have comparable activation variance and comparable backpropagated-gradient variance.
- This is a design target based on simplifying assumptions, not an exact guarantee for every trained network.

Slide 51 - Reconciling fan-in and fan-out

```
Forward constraint:  fan_in * Var(W) = 1
Backward constraint: fan_out * Var(W) = 1
Compromise:          Var(W) = 2 / (fan_in + fan_out)
                     SD(W) = sqrt(2 / (fan_in + fan_out))
```

- The forward and backward requirements coincide if `fan_in=fan_out`.
- Otherwise, using their average provides the Glorot compromise.
- Be careful to distinguish variance from standard deviation: only standard deviation takes the square root.

Slide 52 - Xavier/Glorot initialization for tanh

```
Normal target: mean 0, SD = sqrt(2 / (fan_in + fan_out))
Uniform:      [-r, r], r = sqrt(6 / (fan_in + fan_out))
```

- A uniform distribution on `[-r,r]` has variance `r^2/3`; matching the target variance yields the 6 in the bound.

- If both fans equal `m`, the normal SD is `sqrt(1/m)` and uniform bound is `sqrt(3/m)`.
- Tanh has derivative 1 at zero, which supports the lecture's near-center approximation.

Slide 53 - Lecture's sigmoid correction

- Sigmoid's derivative at zero is `1/4`, versus 1 for tanh.
- The lecture multiplies the normal SD and uniform bound from slide 52 by 4 to compensate.
- Multiplying SD by 4 multiplies variance by 16, which explains the next implementation's `scale=16`.
- Treat this as the lecture's sigmoid-specific variant; it is not built into the ordinary Glorot initializer merely by choosing sigmoid activation.

Slide 54 - He initialization for ReLU

- ReLU removes negative outputs. Under a symmetric preactivation distribution, roughly half the units are inactive.
- The lecture compensates with a larger weight variance: multiply the Glorot SD/bound by `sqrt(2)`.

```
Lecture's fan-average version:
SD = sqrt(4 / (fan_in + fan_out))
r  = sqrt(12 / (fan_in + fan_out))
```

- **Implementation distinction:** built-in He initialization uses fan-in: `SD=sqrt(2/fan_in)` and uniform bound `sqrt(6/fan_in)`. The formulas coincide when fan-in equals fan-out. See Keras initializer documentation.
- More precisely, the half-factor argument concerns signal second moments; describing it solely as a reduction in variance is a simplification.

Slide 55 - He initialization reference

- Cites He, Zhang, Ren, and Sun, *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*.
- This is a reference slide with no additional formula.

Slide 56 - Initialization summary table

Let `F=fan_in+fan_out`. The lecture summarizes:

Activation	Uniform half-width r	Normal SD
Tanh	`sqrt(6/F)`	`sqrt(2/F)`
Sigmoid, lecture variant	`4*sqrt(6/F)`	`4*sqrt(2/F)`
ReLU, lecture fan-average variant	`sqrt(12/F)`	`sqrt(4/F)`

- Match initialization to the activation's effect on signal scale.
- Preserve the distinction between these lecture variants and the built-in initializer names.

Slide 57 - Implementing tanh and sigmoid initialization

```
tf.keras.layers.Dense(
    2, activation="tanh", kernel_initializer="glorot_uniform"
)
# Use "glorot_normal" for the normal variant.

sigmoid_init = tf.keras.initializers.VarianceScaling(
    scale=16.0, mode="fan_avg", distribution="uniform"
)
tf.keras.layers.Dense(
    2, activation="sigmoid", kernel_initializer=sigmoid_init
)
```

- `fan_avg` uses the average of input and output fan counts; `scale=16` implements the lecture's fourfold SD correction.
- The slide's untruncated normal variant uses `distribution="untruncated_normal"`.
- **Clarification:** built-in `glorot_normal` samples a truncated normal rather than a literal untruncated Gaussian. See Keras initializer documentation.
- Use a consistent namespace such as `tf.keras`; the slide imports TensorFlow but also uses bare `keras`.

Slide 58 - Implementing He initialization

```
# Built-in fan-in version:
tf.keras.layers.Dense(2, activation="relu", kernel_initializer="he_normal")
# Or kernel_initializer="he_uniform".

# Lecture's fan-average version:
he_avg_init = tf.keras.initializers.VarianceScaling(
```

```
    scale=2.0, mode="fan_avg", distribution="uniform"
)
tf.keras.layers.Dense(2, activation="relu", kernel_initializer=he_avg_init)
```

- The explicit `VarianceScaling` version uses both incoming and outgoing counts.
- The built-in He version uses only incoming counts. For unequal layer widths, these are different initializations.

Slide 59 - Initialization and gradient histograms

- The upper panel uses the preliminary normalized initialization; gradient distributions differ substantially across layers, with early layers having smaller gradients.
- The lower panel uses Glorot uniform; distributions overlap much more closely across layers.
- Similar gradient scales mean earlier layers have a better opportunity to learn.
- The figure illustrates the goal of preserving backward signal, not just keeping forward activations reasonable.

Slide 60 - Outline: solution 2, nonsaturating activations

- Transition to choosing activation functions that maintain useful derivatives.

Slide 61 - Activation choice and gradient flow

- Sigmoid saturates near 0 and 1; tanh saturates near -1 and 1. Both have tiny derivatives for large input magnitudes.
- ReLU and its variants avoid saturation on the positive side and are inexpensive to compute.
- The instructor's reported ranking is `ELU > PReLU > ReLU > tanh > sigmoid`; treat it as classroom experience, not a universal ordering.
- ReLU still has zero slope for negative inputs. The nonsaturation benefit concerns its active positive region.
- Good activation choice and good initialization complement each other.

Slide 62 - Outline: solution 3, batch normalization

- Introduces a technique that manages activation scale during training, beyond the initial weights.

Slide 63 - Batch normalization: motivation

- Weight initialization helps at the start; batch normalization (BN) manages intermediate signals as training changes the weights.
- The lecture motivates BN through **internal covariate shift**: later layers receive changing input distributions as earlier layers learn.
- BN centers and scales each feature using statistics from the current mini-batch.
- It can be placed before or after an activation, as the implementation slides later illustrate.
- Study the normalization mechanism directly; the motivation is the lecture's explanation of why it helps.

Slide 64 - Why learn a scale and shift?

- Normalizing without additional flexibility would constrain the representation each layer can produce.
- BN therefore follows normalization with `y = gamma*x_hat + beta`.
- `gamma` and `beta` are learned, allowing the model to alter or approximately undo normalization when that improves the objective.
- **Clarification:** sigmoid is approximately linear near zero, not exactly linear. Standardizing its inputs does not by itself make an entire neural network unable to represent nonlinear functions.
- The reliable point is that learned scale and shift preserve representational flexibility.

Slide 65 - Normalize each feature across the mini-batch

- For a given node/feature `k`, collect its values over all `m` examples in the batch.
- Compute that feature's batch mean and variance, then normalize all `m` values with those statistics.
- Each feature has its own learned `gamma_k` and `beta_k`.
- BN normalizes a feature across examples, rather than normalizing all features within a single example.
- The transform can recover the original values by choosing the original scale and mean; including epsilon, exact recovery for a fixed batch uses `gamma=sqrt(variance+epsilon)` and `beta=mean`.

Slide 66 - BN parameters and regularization

- Before learned scale and shift, normalized features have mean approximately 0 and variance approximately 1; epsilon makes the variance slightly smaller than 1.
- `gamma` and `beta` are learned by backpropagation.

- Batch means and variances are computed from data, not optimized through a loss as independent parameters.
- Different batches produce different statistics, introducing noise that can have a regularizing effect.
- After scale and shift, outputs need not have mean 0 or variance 1.
- Original paper named in the slide: Ioffe and Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*.

Slide 67 - BN algorithm

For each feature, over a batch of size `m`:

```
mu_B = (1/m) * sum_i x_i
var_B = (1/m) * sum_i (x_i - mu_B)^2
x_hat_i = (x_i - mu_B) / sqrt(var_B + epsilon)
y_i = gamma * x_hat_i + beta
```

- Learn scale `gamma` and shift `beta`; compute batch statistics.
- The variance shown uses division by `m`, not `m-1`.
- `epsilon` avoids division by zero and improves numerical stability.
- For multiple features, the means, variances, scales, and shifts are vectors, with operations applied feature by feature.

Slide 68 - BN at prediction time

- At training time, BN uses the current batch's statistics.
- Prediction batches may be unrepresentative, and a prediction may involve only one example.
- At inference, use stored moving estimates from training rather than recalculating statistics from the test batch.
- **Clarification:** these are estimated training statistics, not exact population means and variances.

Slide 69 - Moving averages for inference

```
mu_moving_new = alpha * mu_moving_old + (1 - alpha) * mu_B
var_moving_new = alpha * var_moving_old + (1 - alpha) * var_B
```

- The slide suggests `alpha=0.99`.

- A high coefficient puts more weight on existing estimates and smooths changes over many batches.
- Moving mean starts at zero and moving variance at one.
- This BN momentum smooths statistics; it is distinct from optimizer momentum that affects parameter updates.

Slide 70 - BN inference formula

```
x_hat_i = (x_i - mu_moving) / sqrt(var_moving + epsilon)
y_i = gamma * x_hat_i + beta
```

- Use stored moving statistics and learned scale/shift.
- No new batch statistics are needed to normalize a single observation.
- The slide's list mentions `sigma_B`, but the prediction formula uses the stored moving variance.

Slide 71 - BN training versus testing summary

Mode	Statistics used	Update moving statistics?
Training	Current mini-batch mean and variance	Yes
Inference	Stored training moving mean and variance	No

- The learned scale and shift are used in both modes.
- This difference explains why `training=True` and `training=False` can produce different outputs. See Keras BatchNormalization documentation.

Slide 72 - BN after hidden activations

- Example architecture: `Input -> BN -> Dense(ELU) -> BN -> Dense(ELU) -> BN -> Dense(linear)`.
- The hidden activations are inside their dense layers, so each following BN receives activated values.
- The input has 2 features, each hidden layer has 2 units, and the regression output has 1 unit.
- The Functional API explicitly connects each layer's output to the next layer's input.

Slide 73 - Counting BN parameters

- Each normalized feature stores four values: learned scale, learned shift, moving mean, and moving variance.
- With two features, each BN layer stores 8 values: 4 trainable and 4 nontrainable.
- Three such layers have 24 stored values: 12 trainable and 12 nontrainable.
- The two hidden dense layers add `2*2+2=6` parameters each; the output adds `2*1+1=3`.
- Total in the displayed model: **39 parameters = 27 trainable + 12 nontrainable**.
- **Clarification:** the stored scale statistic is moving variance, not a freely learned standard deviation. Nontrainable statistics still change during training through their moving-average updates.

Slide 74 - BN before the activation

- Alternative ordering: `Dense(linear) -> BN -> Activation(ELU)`.
- Remove `activation="elu"` from `Dense` and add a separate activation layer after BN.
- When BN immediately follows the linear dense layer, `use_bias=False` can remove a redundant dense bias because BN has a learned offset.
- The lecture recommends trying both placements and selecting based on validation performance.
- Distinguish this ordering from slide 72, where normalization follows the nonlinear activation.

Slide 75 - BN benefits and cost

- The lecture attributes better gradient flow, less initialization sensitivity, and some regularization to BN.
- BN adds computation and stored parameters, so an individual training step can take longer.
- If optimization improves enough to require fewer steps, overall time to reach a given performance can decrease.
- A slower epoch and faster learning are compatible: compare total time to a target validation result.

Slide 76 - Outline: solution 4, gradient clipping

- Introduces bounding large gradients to limit unstable updates.

Slide 77 - Clipping by value

- Exploding gradients can cause enormous updates, infinities, or NaNs.
- Value clipping caps each gradient component independently.

```
optimizer = tf.keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="mse", optimizer=optimizer)
```

- Example: `[0.8, 20.0]` becomes `[0.8, 1.0]`.
- Relative component sizes change, so the gradient direction can change.
- The threshold balances stability against excessive restriction of useful updates. Clipping limits large gradients; it does not restore vanishing ones.

Slide 78 - Clipping by norm

```
If ||d||_2 > c: d_clipped = c * d / ||d||_2
Otherwise:      d_clipped = d
```

- All components of the clipped vector receive the same scaling, preserving that vector's direction.
- With threshold 1, `[0.8,20]` becomes approximately `[0.039968,0.999201]`.

```
optimizer = tf.keras.optimizers.SGD(clipnorm=1.0)
```

- **Implementation detail:** Keras `clipnorm` clips each weight tensor's gradient separately. For one norm over all parameter gradients together, use `global_clipnorm=1.0`. Separate per-tensor scaling need not preserve the direction of the full concatenated gradient. See Keras SGD documentation.

Slide 79 - Outline: avoiding overfitting

- Transition from improving training to improving generalization.

Slide 80 - Why regularization is needed

- Large networks have many parameters and can fit complex patterns.
- The same flexibility allows fitting accidental features of the training set.
- Regularization aims to improve performance on unseen data by constraining or disrupting overfitting.
- Watch validation performance, not just whether training loss decreases.

Slide 81 - Outline: early stopping

- Focuses on using validation performance to decide when to end training.

Slide 82 - Early stopping and patience

- Training error may continue falling while validation error begins rising; the plotted gap illustrates overfitting.
- Retain the model from the best validation point rather than necessarily the final epoch.
- Because validation curves fluctuate, **patience** allows a number of epochs without improvement before stopping.
- The slide suggests 10-20 epochs as possible settings, depending on the dataset and network.
- **Key distinction:** early stopping uses validation data; the final test set remains separate for evaluation.

Slide 83 - Saving and loading models

```
model.save("my_model.keras")
model = tf.keras.models.load_model("my_model.keras")
```

- The saved model includes its structure, weights, and optimizer configuration/state in the lecture's example.
- Saving allows a training script to hand off a model to another script for prediction or later work.
- Model saving by itself does not choose the best epoch or stop training.

Slide 84 - Checkpoint callbacks

```
checkpoint = tf.keras.callbacks.ModelCheckpoint(
    filepath="my_model.keras", save_freq="epoch"
)
model.fit(x_train, y_train, epochs=100, callbacks=[checkpoint])
```

- A callback runs at defined training events; this checkpoint saves at the end of every epoch.
- A fixed path is overwritten by each save, leaving the latest saved model.
- An integer `save_freq` specifies a batch interval.
- Checkpointing protects a saved state; it does not inherently identify the best validation model.

Slide 85 - Save only the best validation model

```
checkpoint = tf.keras.callbacks.ModelCheckpoint(
    "my_model.keras", monitor="val_loss", save_best_only=True
)
model.fit(
    x_train, y_train, validation_data=(x_val, y_val),
    epochs=100, callbacks=[checkpoint]
)
model = tf.keras.models.load_model("my_model.keras")
```

- Save only when the monitored validation result improves.
- Reloading restores the best saved model after the run.
- **Important distinction:** `save_best_only=True` alone still runs all requested epochs. It selects a checkpoint; it does not stop the training loop.
- The slide's `tf.keras.model.load_model` spelling should be `tf.keras.models.load_model`.

Slide 86 - Stop training and restore best weights

```
checkpoint = tf.keras.callbacks.ModelCheckpoint(
    "my_model.keras", monitor="val_loss", save_best_only=True
)
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True
)
model.fit(
    x_train, y_train, validation_data=(x_val, y_val), epochs=100,
    callbacks=[checkpoint, early_stopping],
)
```

- `epochs=100` is now an upper limit; early stopping can end the run sooner.
- `patience=10` tolerates 10 epochs without improvement before stopping.
- `restore_best_weights=True` returns the in-memory model's weights to the best epoch, while checkpointing keeps a saved model for another session.
- Restoring best weights is conceptually different from reloading the full saved model, including optimizer state.

Slide 87 - Early stopping in a manual loop

- The lecture's bonus assignment concerns a custom epoch/instance loop where `fit()` callbacks do not run automatically.
- A manual implementation must track best validation loss, save or copy the best weights, count non-improving epochs, stop at the patience threshold, and restore the best weights.
- Evaluate validation performance consistently after each epoch so comparisons are meaningful.

Slide 88 - Outline: L1 and L2 regularization

- Introduces explicit penalties on weight magnitudes.

Slide 89 - Lasso, ridge, and elastic net

```
L1:          L_total = L_data + lambda * sum_i |theta_i|
L2:          L_total = L_data + (lambda/2) * sum_i theta_i^2
Elastic net: L_total = L_data + r*lambda*sum_i |theta_i|
                       + (1-r)*(lambda/2)*sum_i theta_i^2
```

- L1 is lasso: **Least Absolute Shrinkage and Selection Operator**.
- L2 is ridge; larger weights contribute quadratically to the penalty.
- Elastic net combines both; `r=1` gives L1 and `r=0` gives L2.
- **Clarification:** the L2 penalty uses the squared L2 norm, not the unsquared norm. The factor `1/2` is a mathematical convention.

Slide 90 - Why L1 and L2 behave differently

- Under the lecture's convention, the L2 penalty gradient is `lambda*theta`; the half cancels the derivative's factor of 2.
- For a nonzero weight, the L1 penalty subgradient is `lambda*sign(theta)`.
- L1 pushes with constant magnitude and favors sparsity; L2 pushes more strongly on larger weights and usually shrinks weights smoothly.
- Example penalty shapes: at `theta=0.1`, `|theta|=0.1` and `theta^2=0.01`; at 3, they are 3 and 9. These are penalty values, not actual update sizes.
- Feature scale matters: large-scale inputs may require smaller coefficients and thus incur smaller penalties. Scale numeric inputs before comparing or penalizing their weights.
- **Clarification:** exact zeros depend on the optimization method; an ordinary gradient update can cross zero rather than land exactly on it.

Slide 91 - Weight penalties in TensorFlow

```
tf.keras.layers.Dense(
    10, activation="elu", kernel_initializer="he_normal",
    kernel_regularizer=tf.keras.regularizers.l1(0.01),
)
# Alternatives:
l2_penalty = tf.keras.regularizers.l2(0.01)
elastic_penalty = tf.keras.regularizers.l1_l2(l1=0.01, l2=0.01)
```

- `kernel_regularizer` applies the penalty to the dense weight matrix.
- The elastic-net example sets two separate penalty coefficients rather than an explicit mixing parameter.
- **Coefficient convention:** the lecture writes L2 with `lambda/2`. Keras L2 uses its supplied coefficient on the squared-weight sum, so matching a mathematical lambda requires accounting for the factor of two.

Slide 92 - Outline: dropout

- Introduces random removal of activations during training.

Slide 93 - What dropout does

- At each training step, selected input or hidden activations are set to zero with probability `p`.
- The mask is temporary; an activation dropped on one step may be active on the next.
- The lecture suggests dropout rates of 10%-50% as common examples.
- Ordinary inference keeps all activations; dropout is disabled.
- The figure contrasts a complete network with a thinned network.
- Dropout can slow convergence while reducing overfitting. Original paper named: Srivastava, Hinton, and colleagues, *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*.

Slide 94 - Dropout as an ensemble

- If `n` activations can each be kept or dropped, there are `2^n` possible masks/subnetworks.
- Training samples subnetworks that share weights rather than separately training every possible network.
- Using the full network at prediction time approximates a combination of these subnetworks.

- **Clarification:** masks can repeat. A run with 100,000 iterations samples 100,000 masks, but does not guarantee 100,000 unique networks.
- This is an ensemble analogy, not an exact average of independently trained models.

Slide 95 - Dropout reduces co-adaptation

- The soccer analogy describes teammates learning to cover for missing players.
- Likewise, units cannot rely on one particular neighboring activation always being present.
- This encourages useful representations that are more robust to missing signals.
- The idea is to reduce excessive reliance on a few inputs or tightly coordinated units, which can reduce overfitting.

Slide 96 - Keep probability and inverted dropout

- Drop probability is `p`; keep probability is `q=1-p`.
- Without compensation, expected signal during training would be smaller than during inference.
- Two conventions can compensate: scale down at inference, or scale retained activations up during training.
- Keras uses **inverted dropout**: kept activations are divided by `q` during training; ordinary inference needs no manual scaling.

```
Training activation = mask * activation / q
E[training activation] = q * activation / q = activation
```

- With `p=0.5`, kept activations double during training. With `p=0.2`, they are multiplied by `1.25`.

Slide 97 - Practical dropout guidance

- The lecture advises against dropout after the output layer, where no later layer can compensate.
- If the model already underfits, dropout can further reduce its effective capacity and hurt performance.
- With limited training time, slower convergence can outweigh the eventual generalization benefit.
- Increase dropout when overfitting; decrease it when underfitting. Consider applying it only after the last few hidden layers.
- Treat these as tuning guidelines based on validation behavior. Early stopping and dropout can be combined; the lecture's concern is stopping before the model has learned adequately.

Slide 98 - Dropout implementation and Monte Carlo dropout

```
dropout_layer = tf.keras.layers.Dropout(0.5)
# Ordinary prediction: dropout is disabled.
# Lecture's stochastic prediction call:
y_hat = model(X, training=True)
```

- Monte Carlo (MC) dropout leaves dropout active at prediction time and runs the same observation through the model repeatedly.
- Average the predictions for a combined estimate; their spread provides a measure of variation under the dropout masks.
- The lecture connects MC dropout with approximate Bayesian inference through Gal and Ghahramani's work.
- **Implementation caution tied to the next slide:** `training=True` also activates training behavior in BN layers, using batch statistics and potentially updating moving statistics. In a model containing BN, enabling dropout alone requires more selective handling. See Keras BN documentation.
- Referenced class example: `dropout.py`.

Slide 99 - Batch normalization and dropout together

- The lecture notes that combining them can worsen performance and cites a paper on their interaction.
- Practical study point: stacking regularization/stabilization methods does not guarantee a better model.
- Compare the combination against simpler alternatives using validation performance.
- This slide does not establish that BN and dropout must never be combined.

Slide 100 - Outline: data augmentation

- Introduces expanding training variation through realistic transformations of existing observations.

Slide 101 - Data augmentation

- Generate additional training instances from existing ones to expose the model to more variation and reduce overfitting.
- The generated examples should remain realistic and retain appropriate labels.
- The lecture emphasizes image/CNN applications and generating examples on the fly during training.

- Its warning about white noise means arbitrary corruption is not a substitute for realistic, useful transformations; the slide's categorical statement that noise cannot help is too broad.
- Example intuition: a transformed image should still represent the original class; transformations that change the label's meaning are unsuitable.

Slide 102 - Outline: depth versus width

- Final topic: the different roles of adding layers and adding units within layers.

Slide 103 - Why depth can help

- Successive nonlinear layers can build progressively more abstract representations.
- Earlier learned features become inputs for later combinations, allowing complex concepts to be composed from simpler ones.
- The lecture compares this with learning basic concepts before higher-level abstractions.
- It cites historical growth from AlexNet-scale depth to very deep residual networks; these are historical examples, not current architecture rankings.
- The core benefit is hierarchical feature construction. Greater depth alone does not guarantee better generalization on every problem.

Slide 104 - Visual hierarchy of learned features

- The illustration progresses from lines/edges in shallow layers, to eyes/noses/ears in middle layers, to facial structure in deeper layers.
- Later features are combinations of earlier features rather than unrelated additions.
- This gives an intuitive explanation of compositional representation learning in an image classifier.
- The pictured hierarchy is illustrative; exact learned features depend on the model and data.

Slide 105 - Why width can help

- More units within a layer increase capacity to represent patterns and frequent feature co-occurrences; the lecture calls this **memorization**.
- Wider layers offer more parallel computation, whereas additional layers create sequential dependencies.
- Excess width can increase parameter count and require stronger regularization.
- The lecture contrasts width's memorization with depth's composition of features and ability to handle new feature combinations.

- These are useful tendencies, not mutually exclusive abilities: both depth and width affect representation, optimization, and generalization.

Review comparisons

Method	What it changes	Main point to remember
Momentum	Smoothed update direction	Retains past movement
Nesterov	Where the gradient correction is evaluated	Looks ahead along momentum
Learning rate schedule	Shared base learning rate over time	Later steps can become smaller
AdaGrad	Per-parameter scaling from all squared gradients	Never forgets; may stall
RMSprop	Per-parameter scaling from recent squared gradients	Exponentially forgets old history
Adam	First and second moments, with bias correction	Momentum plus adaptive scaling
Initialization	Starting weight distribution	Preserve forward/backward signal scale
BN	Intermediate signal normalization	Batch statistics in training; moving statistics in inference
Clipping	Excessive gradient magnitude	Value clipping may rotate; norm clipping rescales
Early stopping	Duration and selected weights	Monitor validation, retain best epoch
L1	Absolute-weight penalty	Encourages sparsity
L2	Squared-weight penalty	Shrinks large weights more strongly
Dropout	Random training activation masks	Keep probability is `1-p`; scale retained outputs
Augmentation	Training examples	Add realistic, label-preserving variation

Self-check questions

1. Why does multiplying a one-hot vector by an embedding table select one row? What extra operation is needed for K-hot equivalence?
2. How does backpropagation differ from an optimizer?
3. What history does momentum keep, and what does Nesterov change?
4. How do optimizer steps differ from epochs in a learning-rate schedule?

5. Why can AdaGrad stall, and how does RMSprop address that?
6. What are Adam's two moments? Why must bias-correction indexing start at 1?
7. How can a high fan-in cause saturation with poorly scaled initial weights?
8. What is the difference between a variance, a standard deviation, and a uniform half-width in initialization formulas?
9. How does lecture fan-average He initialization differ from built-in `he_normal`?
10. Which BN values are learned, and which are computed? What changes at prediction time?
11. Why can value clipping change direction while norm clipping preserves a vector's direction?
12. Why does saving the best checkpoint not automatically stop training?
13. How do L1 and L2 penalty derivatives differ?
14. With dropout rate 0.25, what is the keep probability and training-time scaling? (Answer: 0.75 and 4/3.)
15. Why can `model(X, training=True)` be problematic for MC dropout when the model also contains BN?
16. How do depth and width differ in feature construction and computational parallelism?

Source and verification notes

Primary source: `deep.pdf`, Michel Ballings, *BZAN 554: Deep Learning for Business Applications - Training Deep Neural Networks*, 105 pages, PDF dated March 2, 2026.

All slides were reviewed using extracted text and rendered pages. Mathematical clarifications separate penalty values from derivatives, variance from standard deviation, and raw second moments from variance. Implementation distinctions are supported by the official Keras documentation linked beside the relevant notes.

The deck references `embedding.py`, `backprop.pdf`, and `dropout.py`; these separate materials were not supplied as part of this request. Notes explain what appears in this deck without claiming to summarize those external class files.